



DOCKER TUTORIAL

Getting Started with Containers

A practical introduction to images, containers, Dockerfiles, and the core workflow you'll use every day.

From zero to your first container

What We'll Cover

Six steps from concept to a running container



1. What is Docker?

The problem it solves and why it matters



2. Containers vs. VMs

How containers differ from virtual machines



3. Core Concepts

Images, containers, Dockerfiles & registries



4. Architecture & Install

How the pieces fit together, and setup



5. Commands & Dockerfile

The hands-on workflow you'll use daily



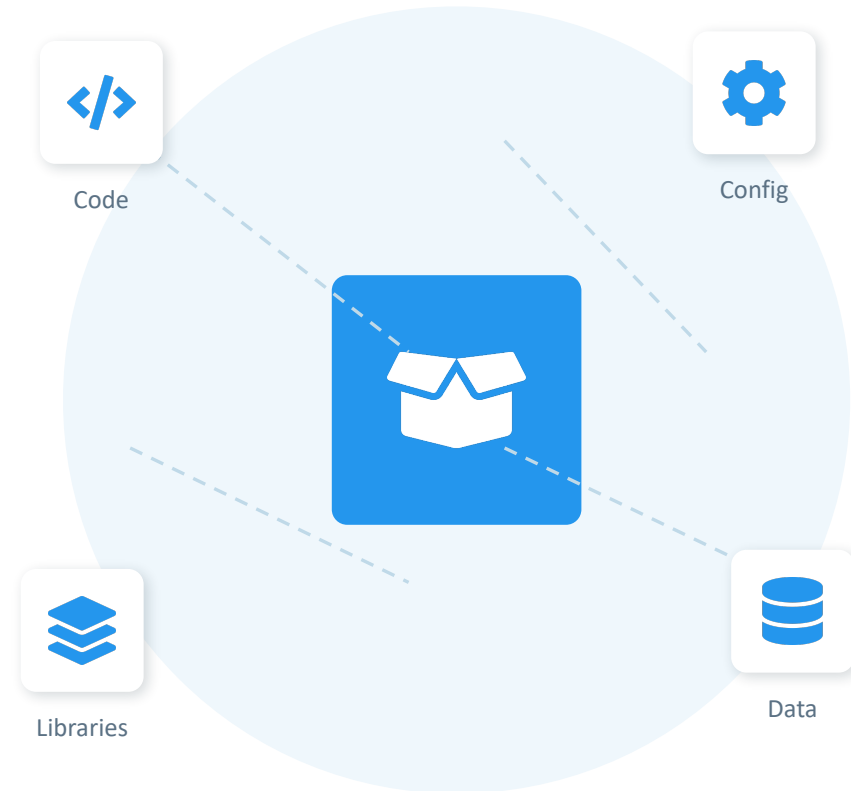
6. Compose & Best Practices

Multi-container apps and good habits

What is Docker?

Docker is an open platform for building, shipping, and running applications inside **containers** — lightweight, isolated environments.

- Packages your app with everything it needs to run: code, runtime, libraries, and system tools
- That package runs the same way on your laptop, a teammate's machine, or in production
- Eliminates the classic "it works on my machine" problem

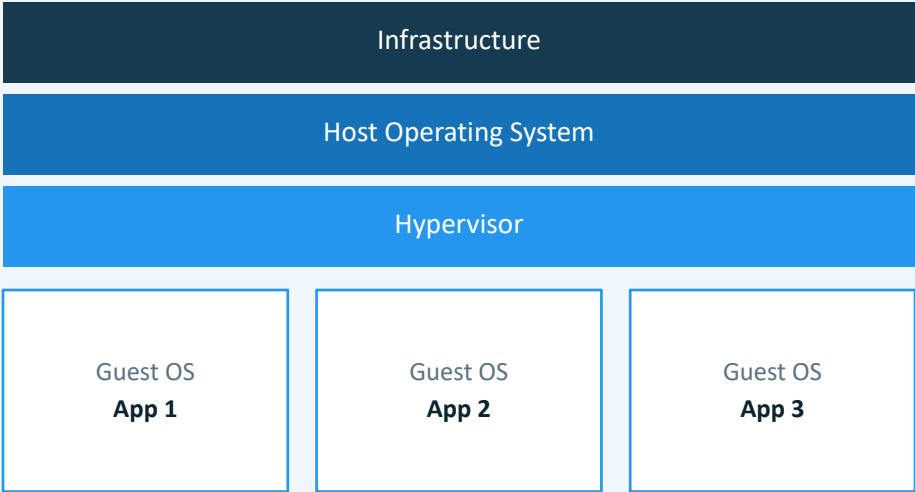


Containers vs. Virtual Machines

Containers share the host OS kernel — VMs each carry a full guest OS



Virtual Machines



Minutes

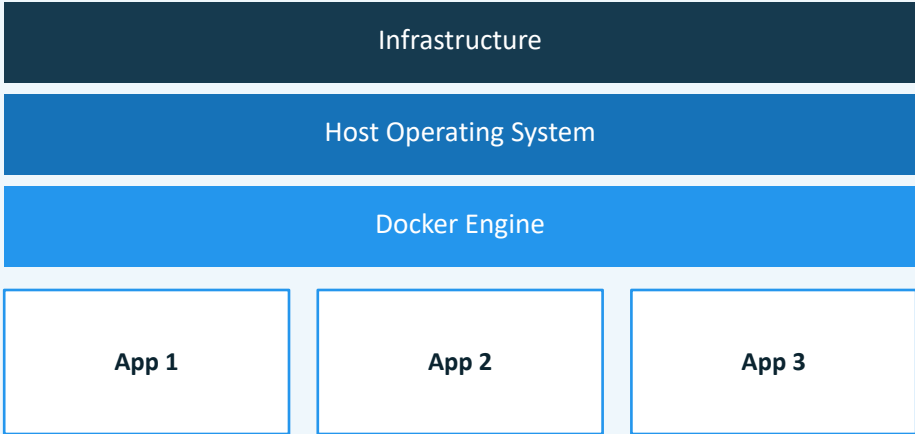
Boot time

GBs

Typical size



Containers (Docker)



Seconds

Boot time

MBs

Typical size

Core Concepts

Four terms you'll see constantly



Image

A read-only template with everything needed to run an app — code, dependencies, and instructions.



Container

A running (or stopped) instance of an image. Isolated, lightweight, and disposable.



Dockerfile

A plain-text file listing the steps Docker follows to build an image.



Registry

A place to store and share images — Docker Hub is the most common public registry.

Docker Architecture

Three pieces work together every time you run a command



The daemon does the real work; the client and registry are how you talk to it.

Installing Docker

Docker Desktop covers Windows and Mac; Linux installs the Engine directly



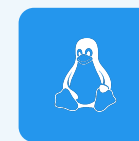
Windows

Install Docker Desktop. Requires WSL 2 (Windows Subsystem for Linux).



macOS

Install Docker Desktop. Available for both Intel and Apple Silicon.



Linux

Install Docker Engine via your package manager (apt, dnf, etc.).

Download: docker.com/get-started



Verify your installation

Open a terminal and run the command below — it should print a version number.

```
$ docker --version
```

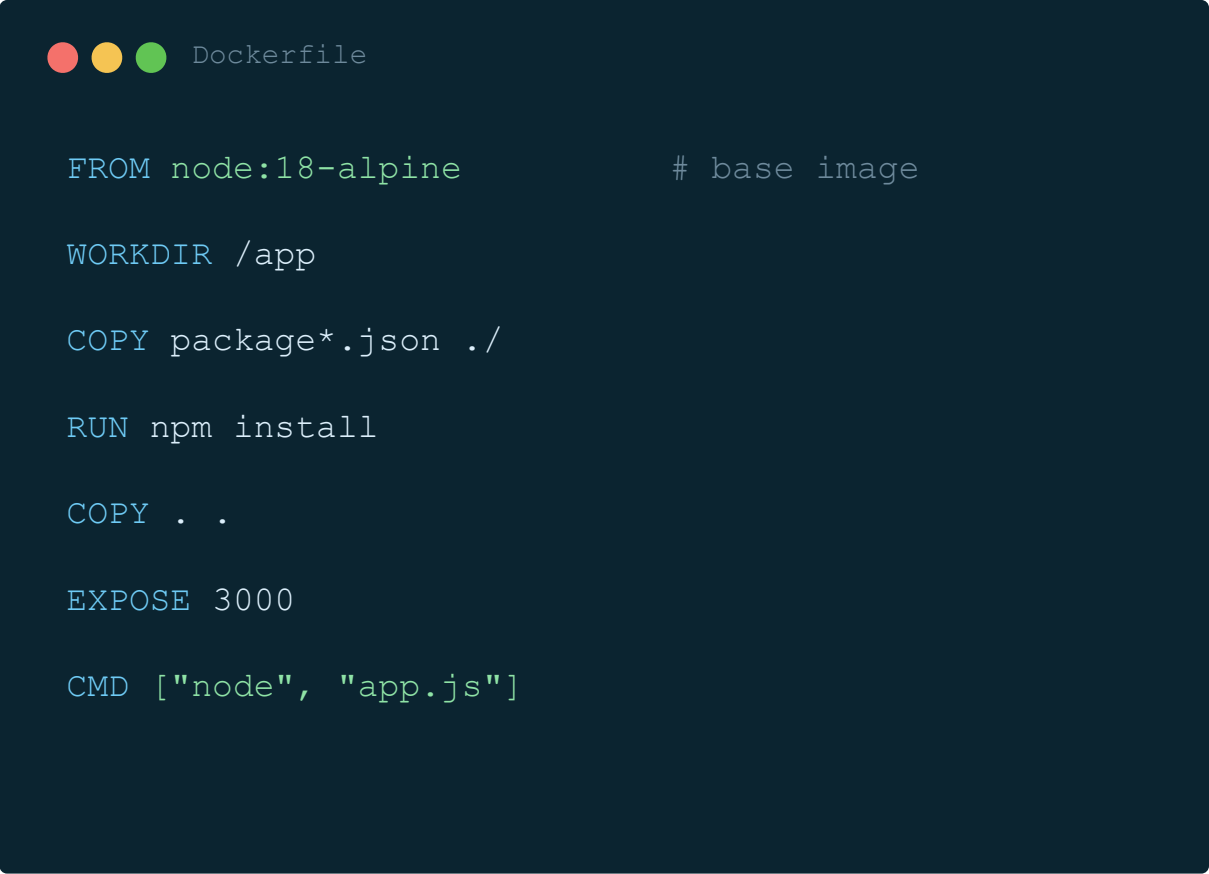
Essential Commands

The handful of commands you'll reach for constantly

COMMAND	WHAT IT DOES
<code>docker pull <image></code>	Download an image from a registry
<code>docker images</code>	List images stored on your machine
<code>docker run <image></code>	Create and start a new container from an image
<code>docker ps</code>	List currently running containers
<code>docker ps -a</code>	List all containers, including stopped ones
<code>docker stop <container></code>	Gracefully stop a running container
<code>docker rm <container></code>	Remove a stopped container
<code>docker rmi <image></code>	Remove an image you no longer need

Writing a Dockerfile

A simple recipe Docker follows to build your image



```
FROM node:18-alpine          # base image

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "app.js"]
```

FROM

Sets the base image to start from

WORKDIR

Sets the working directory inside the container

COPY

Copies files from your machine into the image

RUN

Executes a command while building the image

EXPOSE

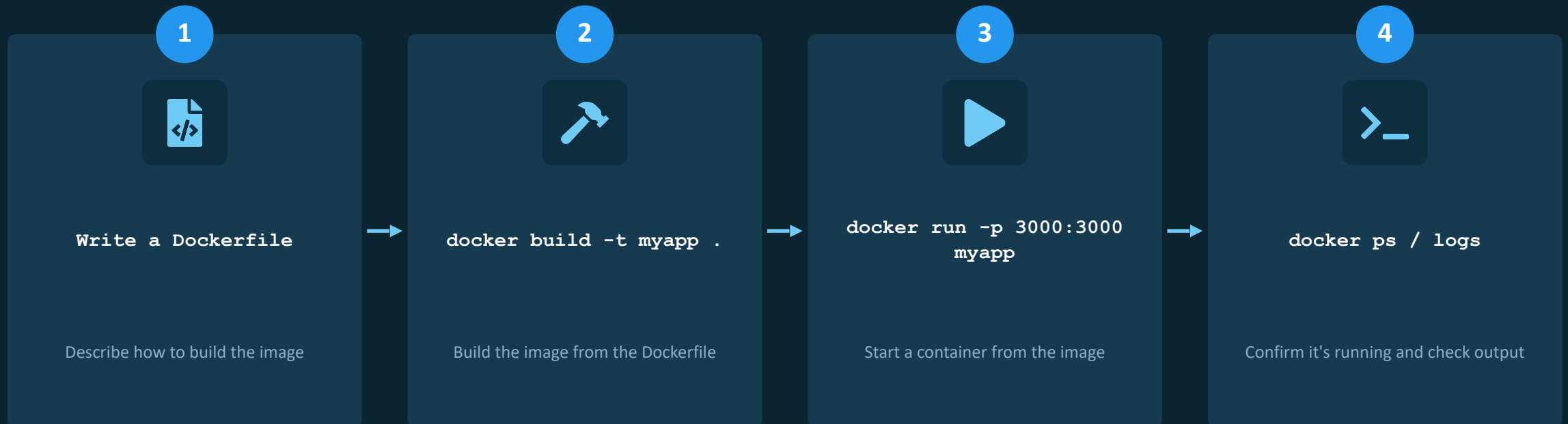
Documents which port the app listens on

CMD

Default command to run when the container starts

Build & Run Workflow

Four steps from source code to a running container



Multi-Container Apps with Compose

Define your whole stack in one file, start it with one command

Real applications usually need more than one container — for example, a web app plus a database.

Docker Compose lets you define every service in a single YAML file.

- One file describes every service, its image, ports, and settings
- One command starts (or stops) the entire application
- Services can talk to each other by name on a shared network

```
$ docker compose up
```

```
docker-compose.yml

services:

  web:

    build: .

    ports:

      - "3000:3000"

  db:

    image: postgres:15

    environment:

      POSTGRES_PASSWORD: example
```

Best Practices

Habits that keep images small, secure, and easy to maintain



Use small base images

Prefer alpine or slim variants to keep images lean and fast to pull.



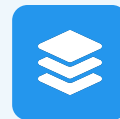
Tag images meaningfully

Use versioned tags (e.g. v1.2.0) instead of relying only on latest.



Avoid running as root

Create and switch to a non-root user inside the container for safety.



One process per container

Keep containers focused — compose multiple containers for multiple jobs.



Add a .dockerignore

Exclude node_modules, .git, and build artifacts from the build context.



Clean up regularly

Run docker system prune to remove unused images and containers.



You're Ready to Containerize

A quick recap before you build your first image

- Docker packages apps into portable, consistent containers
- Images are templates; containers are running instances of them
- A Dockerfile defines how an image is built, step by step
- Docker Compose runs multi-service apps with one command



Keep learning: docs.docker.com · hub.docker.com